

Apostila de Introdução à Programação e Algoritmos

Sumário

1. Introdução à Programação

1.1 O que é programar?

1.2 O que é um algoritmo?

1.3 Características de um bom algoritmo

2. Representação de Algoritmos

2.1 Linguagem natural

2.2 Pseudocódigo

2.3 Fluxogramas

3. Conceitos Básicos de Programação

3.1 Tipos de dados primitivos

3.2 Constantes

3.3 Variáveis

3.4 Atribuições

4. Operadores

4.1 Operadores aritméticos

4.2 Operadores relacionais

4.3 Operadores lógicos

4.4 Precedência de operadores

5. Comandos de Entrada e Saída

5.1 Entrada de dados

5.2 Saída de dados

6. Estruturas de Condição

6.1 Estrutura de condição simples (SE-ENTÃO)

6.2 Estrutura de condição composta (SE-ENTÃO-SENÃO)

6.3 Estrutura de condição encadeada

6.4 Estrutura de condição do tipo "CASO SEJA" (escolha)

7. Estruturas de Repetição

7.1 Repetição com teste no início (ENQUANTO)

7.2 Repetição com teste no final (REPITA-ATÉ)

7.3 Repetição contada (PARA)

8. Exercícios Propostos (sugestões)

1. Introdução à Programação

1.1 O que é programar?

Programar é o ato de escrever instruções para que o computador execute tarefas específicas.

Essas instruções são escritas em uma linguagem de programação, que o computador consegue interpretar e executar.

Exemplos de tarefas:

- Calcular a média de notas de uma turma.
- Controlar o caixa de um supermercado.
- Gerenciar o estoque de uma loja.
- Exibir um site, um jogo ou um aplicativo.

1.2 O que é um algoritmo?

Um algoritmo é uma sequência finita de passos, bem definidos, que descrevem como resolver um problema.

Em outras palavras:

> Algoritmo = receita de bolo para resolver um problema.

Exemplo (em linguagem natural) - algoritmo para fazer um café:

1. Colocar água na chaleira.
2. Ligar o fogo e aquecer a água.
3. Colocar o pó de café no filtro.
4. Quando a água ferver, desligar o fogo.
5. Despejar a água quente sobre o pó no filtro.
6. Aguardar o café coar.
7. Servir o café na xícara.

1.3 Características de um bom algoritmo

Um bom algoritmo deve ser:

- Finito: termina após um número limitado de passos.
- Definido: cada passo é claro, sem ambiguidades.
- Ordenado: os passos têm uma sequência lógica.
- Eficiente: resolve o problema com o menor esforço possível (tempo e recursos).

2. Representação de Algoritmos

Para sair da ideia e chegar na programação, representamos algoritmos de algumas formas:

- Em linguagem natural
- Em pseudocódigo
- Em fluxogramas

2.1 Linguagem natural

É a forma mais simples de descrever um algoritmo, usando frases em português (ou outro idioma humano).

Vantagem:

- Fácil de entender por qualquer pessoa.

Desvantagem:

- Pode gerar ambiguidades, pois a linguagem humana nem sempre é precisa.

Exemplo - algoritmo para somar dois números:

1. Pedir o primeiro número ao usuário.
2. Pedir o segundo número ao usuário.
3. Somar os dois números.
4. Mostrar o resultado da soma.

2.2 Pseudocódigo

Pseudocódigo é uma forma de escrever algoritmos usando uma mistura de linguagem natural com uma estrutura próxima das linguagens de programação, mas sem seguir a sintaxe rígida de nenhuma linguagem específica.

Objetivo:

- Ser simples de entender;

- Estar próximo do código que será implementado depois.

Exemplo (somar dois números):

```
``text
INÍCIO
LER A
LER B
SOMA A + B
ESCREVER "A soma é: ", SOMA
FIM
---
```

Neste exemplo:

- `LER` representa a entrada de dados.
- `ESCREVER` representa a saída de dados.
- `` representa a atribuição (recebe).

2.3 Fluxogramas

Fluxograma é uma representação gráfica de um algoritmo, usando símbolos para representar ações e setas para indicar o fluxo.

Principais símbolos (descrição textual):

- Elipse / oval: Início e Fim
- Retângulo: processo (cálculo, atribuição, ação)
- Paralelogramo: entrada/saída de dados
- Losango: decisão (condição com resposta SIM/NÃO, VERDADEIRO/FALSO)
- Setas: indicam a direção do fluxo

Exemplo (descrição em texto) - fluxograma para somar dois números:

1. [Início]
2. [Entrada] Ler A
3. [Entrada] Ler B
4. [Processo] SOMA A + B
5. [Saída] Escrever SOMA
6. [Fim]

3. Conceitos Básicos de Programação

3.1 Tipos de dados primitivos

Em programação, dados são informações que o programa manipula. Cada dado tem um tipo, que define o que ele representa e como pode ser usado.

Tipos primitivos mais comuns:

- Inteiro (int)

Números inteiros, positivos ou negativos, sem parte decimal.

Exemplos: -10, 0, 5, 200

- Real (float / double)

Números com parte decimal.

Exemplos: 3.14, -0.5, 10.0

- Caractere (char)

Um único símbolo (letra, número, sinal).

Exemplos: 'A', '5', '#'

- Cadeia de caracteres (string)

Uma sequência de caracteres (texto).

Exemplos: "Maria", "Rua 10", "Olá mundo"

- Lógico / Booleano (bool)

Representa verdade ou falsidade.

Valores possíveis: VERDADEIRO ou FALSO (TRUE ou FALSE)

Em pseudocódigo, podemos declarar assim (estilo genérico):

```
``text
INTEIRO idade
REAL salario
CARACTERE letra
CADEIA nome
LÓGICO aprovado
````
```

### 3.2 Constantes

Uma constante é um valor que não se altera durante a execução do programa.

Exemplos:

- O valor de PI (3,14159)
- A quantidade de dias da semana (7)

Em pseudocódigo:

```
```text
CONSTANTE REAL PI 3.14159
CONSTANTE INTEIRO DIAS_SEMANA 7
```
```

Você pode usar constantes para representar valores fixos, facilitando a leitura e manutenção do código.

### 3.3 Variáveis

Uma variável é um espaço na memória do computador onde podemos armazenar dados que podem mudar ao longo do programa.

- Tem um nome (identificador)
- Tem um tipo (inteiro, real, lógico, etc.)
- Tem um valor (que pode ser alterado)

Regras comuns para nomes de variáveis:

- Não começar com número;
- Não conter espaços;
- Evitar acentos;
- Deve ser significativa (ex.: `idade`, `salario`, e não `x`, `y` sempre).

Exemplos de declaração em pseudocódigo:

```
```text
INTEIRO idade
REAL salario
CADEIA nome
LÓGICO maior_de_idade
```
```

### 3.4 Atribuições

Atribuição é a operação de guardar um valor em uma variável.

Símbolos comuns:

- `` (recebe) em pseudocódigo
- `=` em muitas linguagens reais (C, Java, Python, etc.)

Exemplos:

```
```text
idade 20
salario 1500.50
nome "Ana"
maior_de_idade VERDADEIRO
```
```

Também podemos atribuir o resultado de expressões:

```
```text
soma A + B
media (nota1 + nota2) / 2
```

```

## 4. Operadores

Operadores são símbolos usados para realizar operações sobre dados.

### 4.1 Operadores aritméticos

Usados para cálculos numéricos.

- `+` adição
- `-` subtração
- `\*` multiplicação
- `/` divisão
- `DIV` divisão inteira (quando disponível)
- `MOD` resto da divisão inteira (módulo)

Exemplos:

```
```text
soma 10 + 5 // soma = 15
dif 10 - 5 // dif = 5
prod 10 * 5 // prod = 50
```

```
quoc 10 / 4 // quoc = 2.5 (se REAL)
parte_inteira 10 DIV 4 // resultado: 2
resto 10 MOD 4 // resultado: 2
...
```

4.2 Operadores relacionais

Usados para comparar valores. O resultado é sempre VERDADEIRO ou FALSO.

- `=` igual
- `<>` diferente
- `>` maior que
- `<` menor que
- `>=` maior ou igual
- `<=` menor ou igual

Exemplos:

```
```text
10 = 5 // FALSO
10 <> 5 // VERDADEIRO
10 > 5 // VERDADEIRO
10 < 5 // FALSO
10 >= 10 // VERDADEIRO
5 <= 10 // VERDADEIRO
...

```

Usados em condições:

```
```text
SE idade >= 18 ENTÃO
maior_de_idade VERDADEIRO
FIMSE
...

```

4.3 Operadores lógicos

Usados para combinar expressões lógicas (verdadeiras/falsas).

- `E` (AND) - verdadeiro se todas forem verdadeiras
- `OU` (OR) - verdadeiro se pelo menos uma for verdadeira

- `NÃO` (NOT) - inverte o valor lógico

Tabela-verdade resumo:

1. Operador E (AND)

A	B	A E B
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	FALSO
FALSO	VERDADEIRO	FALSO
FALSO	FALSO	FALSO

2. Operador OU (OR)

A	B	A OU B
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

3. Operador NÃO (NOT)

A	NÃO A
VERDADEIRO	FALSO
FALSO	VERDADEIRO

Exemplos em condições:

```text

SE (idade >= 18) E (idade <= 65) ENTÃO

ESCREVER "Adulto"

FIMSE

SE (nota >= 7) OU (trabalho\_extra = VERDADEIRO) ENTÃO

ESCREVER "Aprovado"

FIMSE

SE NÃO (chovendo) ENTÃO

ESCREVER "Pode sair sem guarda-chuva"

FIMSE

...

## 4.4 Precedência de operadores

Precedência define a ordem em que as operações são executadas quando há mais de um operador na mesma expressão.

Ordem típica (da maior para a menor prioridade):

1. Parênteses: `(` `)`
2. Operadores aritméticos: `\*`, `/`, `DIV`, `MOD`, depois `+`, `-`
3. Operadores relacionais: `=`, `<>`, `>`, `<`, `>=`, `<=`,
4. Operadores lógicos: `NÃO`, depois `E`, depois `OU`

Quando houver dúvida, use parênteses para deixar claro.

Exemplos:

```text

resultado 2 + 3 * 4 // resultado = 2 + 12 = 14

resultado (2 + 3) 4 // resultado = 5 4 = 20

condicao (idade > 18) E (idade < 60)

...

5. Comandos de Entrada e Saída

5.1 Entrada de dados

Comandos usados para receber dados do usuário.

Em pseudocódigo, usaremos:

- `LER` para representar entrada de dados.

Exemplo:

```text

INÍCIO

ESCREVER "Digite sua idade: "

LER idade

FIM

...

Explicação:

- `ESCREVER` mostra a mensagem na tela.
- `LER` espera o usuário digitar um valor e armazena na variável `idade`.

## 5.2 Saída de dados

Comandos usados para mostrar informações ao usuário.

Em pseudocódigo, usaremos:

- `ESCREVER` para representar saída de dados.

Exemplos:

```
```text
ESCREVER "Olá, mundo!"
ESCREVER "Sua idade é: ", idade
```
```

Podemos combinar entrada e saída em um mesmo algoritmo:

```
```text
INÍCIO
ESCREVER "Digite seu nome: "
LER nome
ESCREVER "Olá, ", nome
FIM
```
```

---

## 6. Estruturas de Condição

As estruturas de condição permitem que o programa tome decisões, executando ou não certos comandos conforme uma condição seja verdadeira ou falsa.

### 6.1 Estrutura de condição simples (SE-ENTÃO)

Forma geral:

```
```text
```

```
SE <condição> ENTÃO
<comandos>
FIMSE
...
```

Se a condição for VERDADEIRA, os comandos dentro do bloco são executados.

Se for FALSA, eles são ignorados.

Exemplo - verificar se a pessoa é maior de idade:

```
```text
INÍCIO
LER idade
SE idade >= 18 ENTÃO
ESCREVER "Maior de idade"
FIMSE
FIM
...

```

## 6.2 Estrutura de condição composta (SE-ENTÃO-SENÃO)

Permite decidir entre dois caminhos: um se a condição for verdadeira, outro se for falsa.

Forma geral:

```
```text
SE <condição> ENTÃO
<comandos-se-verdadeiro>
SENÃO
<comandos-se-falso>
FIMSE
...

```

Exemplo - aprovado ou reprovado:

```
```text
INÍCIO
ESCREVER "Digite a nota: "
LER nota

SE nota >= 7 ENTÃO
ESCREVER "Aprovado"

```

```
SENÃO
ESCREVER "Reprovado"
FIMSE
FIM
...
```

### 6.3 Estrutura de condição encadeada

Quando há mais de duas opções, podemos encadear vários SE-SENÃO.

Forma geral:

```
``text
SE <condição1> ENTÃO
<comandos1>
SENÃO
SE <condição2> ENTÃO
<comandos2>
SENÃO
SE <condição3> ENTÃO
<comandos3>
SENÃO
<comandos4> // caso nenhuma das condições anteriores seja verdadeira
FIMSE
FIMSE
FIMSE
...
```

Muitos pseudocódigos permitem uma forma mais clara (SENÃO SE), mas o encadeamento acima é a ideia básica.

Exemplo - classificação de nota:

```
``text
INÍCIO
LER nota

SE nota >= 9 ENTÃO
ESCREVER "Excelente"
SENÃO
SE nota >= 7 ENTÃO
```

```
ESCREVER "Bom"
SENÃO
SE nota >= 5 ENTÃO
ESCREVER "Regular"
SENÃO
ESCREVER "Reprovado"
FIMSE
FIMSE
FIMSE
FIM
...
```

(Linguagens e notações mais modernas costumam usar `SENÃO SE` para deixar mais limpo, mas conceitualmente é a mesma coisa.)

## 6.4 Estrutura de condição (CASO SEJA) - escolha

A estrutura CASO (também conhecida como switch, escolha, seletor múltiplo) é usada quando queremos escolher uma dentre várias opções baseadas em um único valor.

Forma geral (pseudocódigo genérico):

```
``text
CASO opcao SEJA
1: <comandos para opção 1>
2: <comandos para opção 2>
3: <comandos para opção 3>
OUTROCASO:
<comandos para qualquer outro valor>
FIMCASO
...
```

Exemplo - menu simples:

```
``text
INÍCIO
ESCREVER "1 - Somar"
ESCREVER "2 - Subtrair"
ESCREVER "3 - Sair"
ESCREVER "Escolha uma opção: "
```

LER opcao

CASO opcao SEJA

1:

ESCREVER "Você escolheu SOMAR"

2:

ESCREVER "Você escolheu SUBTRAIR"

3:

ESCREVER "Você escolheu SAIR"

OUTROCASO:

ESCREVER "Opção inválida"

FIMCASO

FIM

---

Vantagens:

- Código mais organizado;
- Melhor leitura quando há muitas opções baseadas em um mesmo valor.

---

## 7. Estruturas de Repetição

Estruturas de repetição (ou laços, loops) permitem executar um conjunto de comandos várias vezes.

Tipos principais:

- Repetição com teste no início (`ENQUANTO`)
- Repetição com teste no final (`REPITA-ATÉ`)
- Repetição contada (`PARA`)

### 7.1 Repetição com teste no início (ENQUANTO)

Enquanto a condição for verdadeira, o bloco de comandos é repetido.

Forma geral:

```
```text
```

```
ENQUANTO <condição> FAÇA
```

```
<comandos>
```

FIMENQUANTO

...

Exemplo - imprimir números de 1 a 5:

```text

INÍCIO

i 1

ENQUANTO i <= 5 FAÇA

ESCREVER i

i i + 1

FIMENQUANTO

FIM

...

Funcionamento:

- Testa `i <= 5`;
- Se verdadeiro, executa o bloco;
- Ao final, incrementa `i` e repete o teste;
- Quando `i` ficar maior que 5, o laço termina.

## 7.2 Repetição com teste no final (REPITA-ATÉ)

Garante que o bloco de comandos seja executado pelo menos uma vez. O teste é feito no final.

Forma geral:

```text

REPITA

<comandos>

ATÉ <condição>

...

Exemplo - ler um número até que ele seja maior que zero:

```text

INÍCIO

REPITA

ESCREVER "Digite um número maior que zero: "

LER numero

ATÉ numero > 0



FIM

...

Funcionamento:

- Executa o bloco (pede e lê o número);
- Testa a condição `numero > 0`;
- Se a condição for falsa, repete;
- Se a condição for verdadeira, sai do laço.

### 7.3 Repetição contada (PARA)

Usada quando sabemos de antemão quantas vezes o laço vai se repetir.

Forma geral:

```
```text
```

```
PARA i DE valor_inicial ATÉ valor_final FAÇA
```

```
<comandos>
```

```
FIMPARA
```

```
```
```

Exemplo - imprimir números de 1 a 10:

```
```text
```

```
INÍCIO
```

```
PARA i DE 1 ATÉ 10 FAÇA
```

```
ESCREVER i
```

```
FIMPARA
```

```
FIM
```

```
```
```

Exemplo - somar 5 números informados pelo usuário:

```
```text
```

```
INÍCIO
```

```
soma 0
```

```
PARA i DE 1 ATÉ 5 FAÇA
```

```
ESCREVER "Digite o ", i, "º número: "
```

```
LER numero
```

```
soma soma + numero
```

```
FIMPARA
```

ESCREVER "A soma dos 5 números é: ", soma

FIM

...

8. Exercícios Propostos (Sugestões)

1. Algoritmo em linguagem natural

Descreva em linguagem natural um algoritmo para:

- Trocar o pneu de um carro;
- Fazer um sanduíche.

2. Pseudocódigo simples

Escreva um algoritmo em pseudocódigo que:

- Leia dois números e mostre a soma;
- Leia dois números e mostre o maior deles.

3. Uso de variáveis e tipos de dados

Declare variáveis adequadas para armazenar:

- Nome de uma pessoa;
- Idade;
- Salário;
- Se ela é casada ou não.

4. Operadores aritméticos

Escreva um algoritmo que leia 3 notas, calcule a média e mostre o resultado.

5. Estrutura SE simples

Algoritmo que leia a idade e, se a pessoa tiver 18 anos ou mais, escreva "Maior de idade".

6. Estrutura SE-SENÃO

Algoritmo que leia uma nota e escreva:

- "Aprovado" se nota ≥ 7 ;
- "Reprovado" caso contrário.

7. Condição encadeada

Algoritmo que leia uma nota e escreva:

- "Excelente" se nota ≥ 9 ;
- "Bom" se nota ≥ 7 e < 9 ;
- "Regular" se nota ≥ 5 e < 7 ;

- "Reprovado" se nota < 5.

8. CASO SEJA (escolha)

Algoritmo que apresente um menu:

1 - Cadastrar

2 - Consultar

3 - Sair

Leia a opção e escreva uma mensagem de acordo com a escolha.

9. Repetição ENQUANTO

Algoritmo que leia números inteiros positivos e some todos eles até que o usuário digite 0.

10. Repetição PARA

Algoritmo que leia 10 números e conte quantos são positivos.

11. Repetição REPITA-ATÉ

Algoritmo que peça ao usuário uma senha numérica até ele digitar o valor correto (por exemplo, 1234).

Esta apostila apresenta os conceitos básicos de algoritmos e programação: representação de algoritmos (linguagem natural, pseudocódigo, fluxograma), tipos de dados, variáveis, operadores, entrada/saída e estruturas de decisão e repetição, com exemplos simples para facilitar o entendimento inicial.